PySpark_Structured_API

Code

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum as sum_
# Create Spark session
spark = SparkSession.builder.appName("RDD_vs_DataFrame").getOrCreate()
# Sample data: list of sales records (RDD)
sales_rdd = spark.sparkContext.parallelize([
("North", 100),
("South", 200),
("North", 150),
("East", 50),
("South", 300)
])
# RDD approach: total sales per region
rdd_totals = (sales_rdd
# .map(lambda x: (x[0], x[1]))
.reduceByKey(lambda a, b: a + b))
print("RDD Totals:", rdd_totals.collect())
```

Output

Code

```
print(sales_rdd)
```

Output

```
ParallelCollectionRDD[18] at readRDDFromFile at PythonRDD.scala:274
```

Code

```
# Dataframe approach
sales_df = spark.createDataFrame(sales_rdd, ["region", "amount"])
df_totals = (sales_df.groupBy("region")
.agg(sum_("amount").alias("total_sales")))
df_totals.show()
```

Error Output

```
[Stage 18:=====================================================>  (71 + 1) / 75]
```

Output

```
+------+-----------+
|region|total_sales|
+------+-----------+
| South|        500|
|  East|         50|
| North|        250|
+------+-----------+
```

Code

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("InMemoryDF").getOrCreate()
data = [
(1, "Alice", 50000),
(2, "Bob", 60000),
(3, "Charlie", 55000)
]
# Implicit schema detection
df = spark.createDataFrame(data, ["id", "name", "salary"])
df.show()
df.printSchema()
```

Output

```
+---+-------+------+
| id|   name|salary|
+---+-------+------+
|  1|  Alice| 50000|
|  2|    Bob| 60000|
|  3|Charlie| 55000|
+---+-------+------+
root
|-- id: long (nullable = true)
|-- name: string (nullable = true)
|-- salary: long (nullable = true)
```

Code

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, DoubleType
spark = SparkSession.builder.appName("ExplicitSchemaDF").getOrCreate()
data = [
(1, "Alice", 50000),
(2, "Bob", 60000),
(3, "Charlie", 55000)
]
schema = StructType([
StructField("id", IntegerType(), False),
StructField("name", StringType(), True),
StructField("salary", IntegerType(), True)
])
df = spark.createDataFrame(data, schema)
df.printSchema()
```

Output

```
root
|-- id: integer (nullable = false)
|-- name: string (nullable = true)
|-- salary: integer (nullable = true)
```

Code

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, DoubleType
spark = SparkSession.builder.appName("DF_from_csv").getOrCreate()
schema = StructType([
StructField("booth_number", IntegerType()),
StructField("booth_price_level", StringType()),
StructField("booth_description", StringType()),
StructField("booth_type", StringType())
])
df = spark.read \
.option("header", "true") \
.schema(schema) \
.csv("booth.csv")
df.show()
df.printSchema()
```

Error Output

```
26/09/04 14:03:51 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/04 14:03:53 WARN Utils: Service 'SparkUI' could not bind on port 4040. Attempting port
4041.
26/09/04 14:03:53 WARN Utils: Service 'SparkUI' could not bind on port 4041. Attempting port
```

```
4042.
26/09/04 14:03:55 WARN Client: Neither spark.yarn.jars nor spark.yarn.archive is set, falling
back to uploading libraries under SPARK_HOME.
[Stage 0:>                                                          (0 + 1) / 1]
```

Output

```
+------------+-----------------+--------------------+----------+
|booth_number|booth_price_level|   booth_description|booth_type|
+------------+-----------------+--------------------+----------+
|           1|                A|First booth to th...|  Standard|
|           2|                A|Second booth to t...|  Standard|
|           3|                B|Third booth to th...|     Small|
|           4|                C|First booth on le...|     Small|
|           5|                C|Second booth on l...|     Small|
|           6|                C|Third booth on le...|     Small|
|           7|                A|First booth to th...|  Standard|
|           8|                C|First booth on ri...|     Small|
|           9|                C|First booth on ri...|     Small|
|          10|                B|Back right booth ...|     Large|
|          11|                B|Back center booth...|     Large|
|          12|                B|Back left booth  ...|     Large|
+------------+-----------------+--------------------+----------+
root
|-- booth_number: integer (nullable = true)
|-- booth_price_level: string (nullable = true)
|-- booth_description: string (nullable = true)
|-- booth_type: string (nullable = true)
```

Code

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("TextFileDF").getOrCreate()
# Reading a text file, each line becomes a row
df_text = spark.read.text("test.txt")
df_text.show()
df_text.printSchema()
```

Output

```
+------+
| value|
+------+
| apple|
|banana|
| apple|
|orange|
|banana|
| apple|
+------+
root
|-- value: string (nullable = true)
```

Code

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("SQLDF").getOrCreate()
df = spark.sql("""
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
""")
df.printSchema()
```

Code

```
from pyspark.sql import SparkSession
# Initialize SparkSession
spark = SparkSession.builder.appName("JSONInferSchema").getOrCreate()
# Read JSON without explicit schema
df_json_infer = spark.read \
.option("multiline", "true") \
.json("filename.json")
# Display DataFrame and schema
df_json_infer.show()
df_json_infer.printSchema()
```

Code

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("JDBCDF").getOrCreate()
jdbc_url = "jdbc:mysql://localhost:3306/employees_db"
table_name = "employee_details"
properties = {"user": "root", "password": "password", "driver": "com.mysql.cj.jdbc.Driver"}
df_jdbc = spark.read.jdbc(url=jdbc_url, table=table_name, properties=properties)
df_jdbc.show()
df_jdbc.printSchema()
```